



Amrita School of AI
COIMBATORE CAMPUS

SESSIONS 4-8

Virtualization and containers

Two ways to run many things on one machine, and they are not the same thing.

Dr. Abhijith Anandakrishnan · Assistant Professor

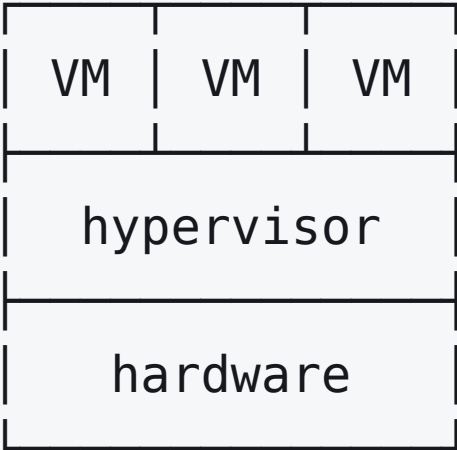
24AIM332 · Odd Semester 2026-27

Cloud exists because one physical machine can pretend to be many.

Resource pooling is not a feature. It is the business model.

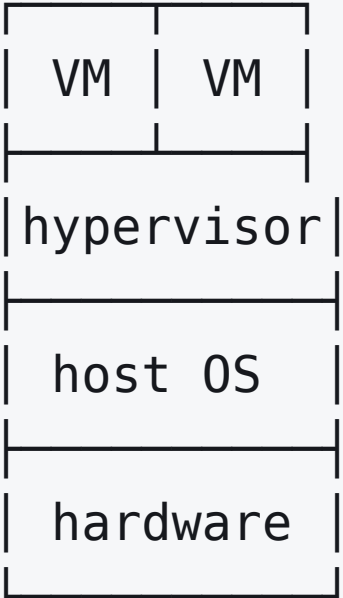
Type 1 and Type 2

TYPE 1 (bare metal)



ESXi, Hyper-V, KVM, Xen

TYPE 2 (hosted)



VirtualBox, VMware Workstation

A full computer, simulated

Each VM has its **own kernel**, its own init, its own filesystem, its own virtual devices.

Boot takes tens of seconds.

Image size: gigabytes.

STRONG ISOLATION

Two VMs share only the hypervisor. A kernel exploit in one does not reach the other.

This is why untrusted multi-tenant workloads still run in VMs.

A container is not a small VM.

It is a process on the host kernel, with a restricted view of the world.

Two Linux kernel features, no magic

Namespaces — what a process can see

pid, net, mnt, uts, ipc, user, cgroup

Its own process tree, its own network interfaces, its own root filesystem.

cgroups — what a process can *use*

CPU shares, memory limit, block I/O, device access.

That is it. There is no container "engine" in the kernel. Docker orchestrates these primitives.

VM vs container

● Virtual machine

- Own kernel
- Boots in ~30 s
- Image in GB
- Isolation: **strong**
- Any OS on any host
- Overhead: a few percent

● Container

- **Shares** the host kernel
- Starts in ~50 ms
- Image in MB
- Isolation: process-level
- Linux on Linux only
- Overhead: near zero

Containers are not a security boundary on their own

A kernel vulnerability is reachable from inside every container on the host.

`--privileged` hands over the machine.

Multi-tenant platforms run containers ****inside VMs****.
You get container speed and VM isolation, and pay for both.

Layers, and why builds are fast

```
FROM python:3.12-slim      # layer 1
WORKDIR /app
COPY requirements.txt .    # layer 2
RUN pip install -r \
    requirements.txt        # layer 3
COPY . .                  # layer 4
CMD ["python", "app.py"]
```

Each instruction makes a **read-only layer**. Layers are content-addressed and cached.

Change your source and only layer 4 rebuilds.

COPY REQUIREMENTS BEFORE SOURCE, ON PURPOSE

Why the COPY order matters

- **COPY .. first**

Any source edit invalidates the cache, so `pip install` runs again.

Every build: minutes.

- **requirements.txt first**

Dependencies are their own layer, cached until they change.

Typical build: seconds.

Small images are not vanity

- Start from `-slim` or `-alpine`
- **Multi-stage builds:** compile in a fat image, copy the binary into a thin one
- `.dockerignore` your `.git` and `node_modules`
- Pin versions; `latest` is not a version

Smaller image → faster pull → faster scale-out → faster deploy → smaller attack surface.

A 1.2 GB image that could be 80 MB costs you on every single node, every time.

Docker in ten lines

```
docker build -t myapp:1.0 .           # build from Dockerfile
docker run -p 8080:80 myapp:1.0       # run, mapping host:container
docker run -d --name api myapp:1.0    # detached, named
docker ps                             # what is running
docker logs -f api                    # follow output
docker exec -it api sh                # shell inside a running container
docker images                         # local images
docker rm -f api                      # stop and remove
docker compose up -d                  # bring up a whole stack
docker compose logs -f                # follow the stack
```

Containers are disposable. Your data is not.

A container's filesystem dies with it. That is the point: it makes containers replaceable.

So anything that must survive goes in a **volume**, or outside the container entirely.

```
docker run -v pgdata:/var/lib/postgresql/data  
postgres
```

Writing a database into the container's own layer is the most common way students lose their data.

One file, a whole stack

```
services:
  web:
    build: .
    ports: ["8080:80"]
    depends_on: [db]
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/pw
    volumes: ["pgdata:/var/lib/postgresql/data"]
volumes:
  pgdata:
```

Good for **one machine**: development, a demo, a small deployment.

It has no idea what to do when the machine dies.

That is the next deck.

Next: many machines, and something that decides where things run.